

Reverse Polish Notation

Infix Notation

Postfix Notation

In Practice

1 + 3 ⇒ 4

1 3 + ⇒ 4

(1 + 3) x 5 ⇒ 20

1 3 + 5 x ⇒ 20

1 + (3 x 5) ⇒ 16

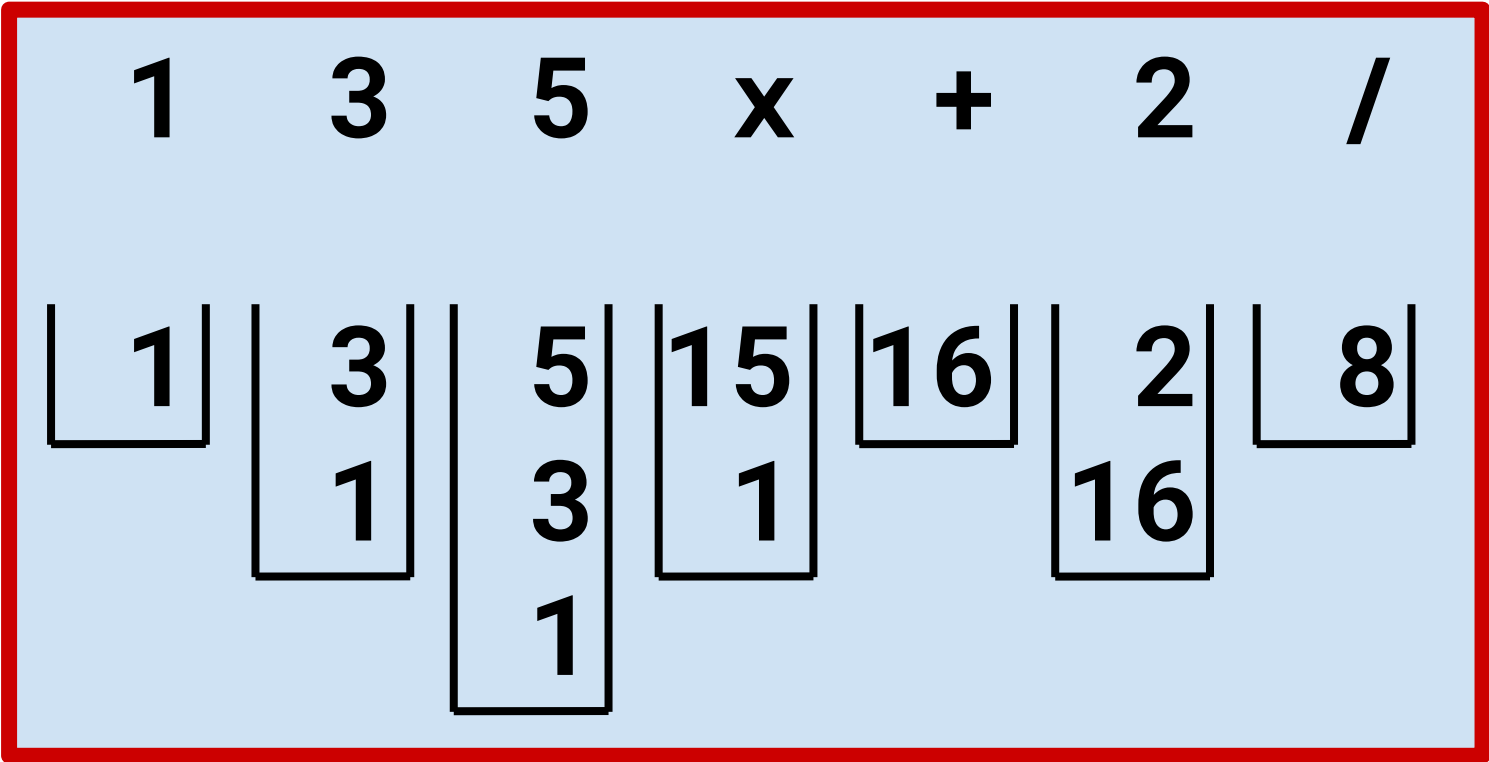
1 3 5 x + ⇒ 16

((1 + 3) x 5) / 2 ⇒ 10

1 3 + 5 x 2 / ⇒ 10

(1 + (3 x 5)) / 2

1 3 5 x + 2 /



No parentheses!

Easy to evaluate!

Turing-complete*!

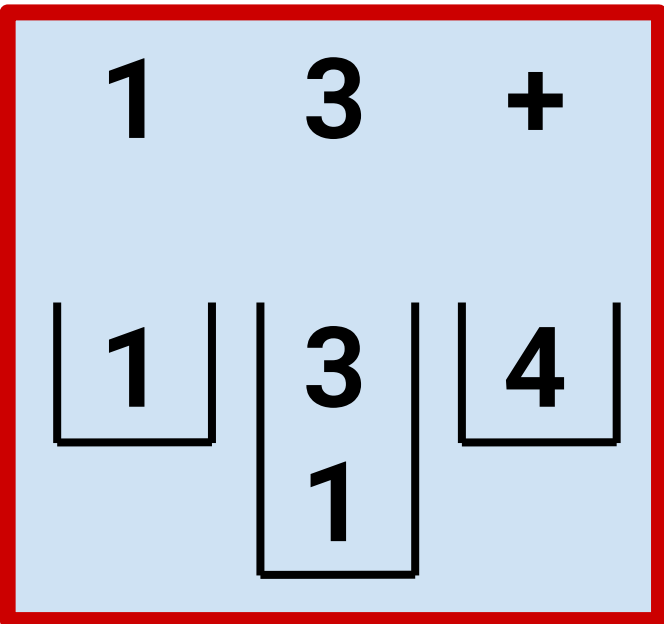
*Clac extension sold separately

Reverse Polish Notation uses a **Stack** to process each expression left-to-right.

Each symbol is either an **operand** (number) or an **operator** (+, -, x, etc).

Operands are pushed to the stack.

Operators take the top two elements off of the stack, operate on them, and put the result back on the stack.



Clac

Clac is an extension to RPN similar to the **Forth** programming language which adds additional operators like *if* and *skip*, as well as custom operators which can be recursive.

This allows “programs” to be written in Clac which are able to perform complex mathematical calculations. It’s actually Turing-complete!

The following implements a Fibonacci using the Clac operators.

: fib dup 2 < if nop 9 skip 1 - dup 1 - fib swap fib + ;

Pseudocode:

```
fib(x):
  if x < 2:
    return x
  return fib(x-2) + fib(x-1)
```

Complete Clac reference reproduced with permission

Before		→	After		Cond/Effect
Stack	Queue		Stack	Queue	
S	n, Q	→	S, n	Q	
S, n	print, Q	→	S	Q	See note #1
S	quit, Q	→	S	Q	See note #2
S, x, y	+, Q	→	S, x + y	Q	See note #3
S, x, y	-, Q	→	S, x - y	Q	See note #3
S, x, y	*, Q	→	S, x × y	Q	See note #3
S, x, y	/, Q	→	S, x / y	Q	See note #4
S, x, y	%, Q	→	S, x % y	Q	See note #4
S, x, y	**, Q	→	S, x ^y	Q	See #3, #4
S, x, y	<, Q	→	S, 1	Q	if x < y
S, x, y	<=, Q	→	S, 0	Q	if x ≥ y
S, x	drop, Q	→	S	Q	
S, x, y	swap, Q	→	S, y, x	Q	
S, x, y, z	rot, Q	→	S, y, z, x	Q	
S, x	if, Q	→	S	Q	if x ≠ 0
S, x	if, tok ₁ , tok ₂ , tok ₃ , Q	→	S	Q	if x = 0
S, x _n , ..., x ₁ , n	pick, Q	→	S, x _n , ..., x ₁ , x _n	Q	See note #5
S, n	skip, tok ₁ , ..., tok _n , Q	→	S	Q	See note #5

Clac should raise an error whenever there are not enough tokens on the stack or the queue for an operation to be performed, or whenever the token on the top of the queue is not one of the ones listed above. Tokens are case sensitive, so **Print** and **PRINT** are not defined, though **print** is.

Notes:

- The **print** token causes n to be printed, followed by a newline.
- The **quit** token causes the interpreter to stop. The **eval** function should then return **false** to indicate that we should just stop, rather than asking for more input.
- This is a 32 bit, two's complement language, so addition, subtraction, multiplication, and exponentiation should behave just as in C0 without raising any overflow errors.
- Division or modulus by 0, or division/modulus of `int_min()` by -1, which would result in an arithmetic error according to the definition of C0 (see page 4 of the [C0 Reference](https://c0.cs.cmu.edu/docs/c0-reference.pdf) at <https://c0.cs.cmu.edu/docs/c0-reference.pdf>), should raise an error in Clac. Negative exponents are undefined and should also raise an error.
- The **pick** token should raise an error if n, the value on the top of the stack, is not strictly positive. The **skip** token should raise an error if n is negative; 0 is acceptable.